

Relatório de Progresso — Trabalho de Conclusão de Curso

Plataforma Distribuída de Monitoramento Ativo da Rede da UFSM

Aluno:	Anthony Carlos Da Silva	Etapa:	Desenvolvimento do MVP (Fases 1–5)
Orientador:	Prof. Carlos Raniery	Data:	17 de agosto de 2026
Curso:	Ciência da Computação — UFSM	Ambiente:	WSL2/Ubuntu (réplica do Raspberry Pi)

1. Resumo executivo

Este relatório descreve o estado atual do desenvolvimento do MVP da plataforma de monitoramento ativo da rede da UFSM. Foram implementadas e **validadas de ponta a ponta** as cinco primeiras fases previstas na proposta: (1) protótipos de medição, (2) agente homogêneo dos probes, (3) orquestração assíncrona via RabbitMQ, (4) controlador central com planos declarativos e (5) ingestão em PostgreSQL/TimescaleDB com visualização em Grafana, incluindo a matriz probe × destino. O núcleo do “resultado mínimo esperado” da proposta já está atingido; restam, para a Fase 6, a operação com múltiplos probes em malha e o controle de concorrência dos testes de vazão (iperf3). Todo o código está versionado no GitHub.

2. Objetivo do trabalho

Projetar e implementar uma arquitetura distribuída de monitoramento ativo baseada em probes homogêneos e reconfiguráveis, capaz de materializar planos declarativos de medição entre pontos internos e destinos externos, consolidando resultados temporais para visualização e análise operacional. O sistema **observa e apresenta** os dados (disponibilidade, latência, perda, vazão, DNS, HTTP/HTTPS, caminho); não infere causa-raiz automaticamente.

3. Arquitetura implementada

A plataforma segue a separação de responsabilidades definida na proposta, com o RabbitMQ como barramento assíncrono e o PostgreSQL/TimescaleDB como fonte definitiva:

Administração → Controlador (FastAPI) → RabbitMQ → Probes (agente Go) → RabbitMQ → Ingestão → PostgreSQL/TimescaleDB → Grafana.

Componente	Tecnologia	Responsabilidade
Agente (probe)	Go	Executa medições, persiste localmente (outbox SQLite), publica resultados
Controlador	Python / FastAPI	Inventário, planos JSON, validação, expansão de malha, publicação de tarefas
Barramento	RabbitMQ	Transporte assíncrono de comandos e resultados (publisher confirms)
Banco	PostgreSQL + TimescaleDB	Fonte de verdade e séries temporais (hypertables)
Visualização	Grafana	Dashboards temporais e matriz probe × destino

4. Decisões técnicas

- **Agente em Go:** binário único para ARM (Raspberry Pi), controle de subprocessos e cliente HTTP nativo com *httptrace* para medir as fases de uma requisição.
- **Controlador em Python/FastAPI:** validação de planos com Pydantic e iteração rápida da API.
- **Contratos de mensagem em JSON Schema** como fonte única, evitando divergência entre o agente (Go) e o controlador (Python).

- **Envelope de resultado comum** + tabelas tipadas por medição (evita modelo EAV; facilita consultas e dashboards).
- **Segurança na execução**: comandos por argumentos explícitos (sem shell), conjunto fechado de tipos de medição e *allowlist* de destinos.
- **iperf3 como medição intrusiva**: dois sentidos modelados como execuções separadas (sem *--bidir*).
- **Identidade permanente do probe** (UUID em arquivo), não derivada de IP/hostname.
- **Tolerância a falha**: persistir no SQLite *antes* de publicar; remover da fila local somente após o *publisher confirm* do broker.

5. O que foi desenvolvido, por fase

Fase 1 — Protótipos de medição

Seis protótipos em Go (ICMP/fping, iperf3, DNS/dig, HTTP com httptrace, traceroute/mtr e inventário do sistema) que emitem um envelope JSON normalizado. **Validado**: cada medição gerou JSON conforme o schema (ex.: ICMP com mínimo/média/máximo, jitter e percentis; HTTP com fases DNS/TCP/TLS/TTFB).

Fase 2 — Agente mínimo

Agente *ufsm-monitor-agent*: configuração por ambiente, executor com timeout, **outbox SQLite (WAL)**, endpoints */health*, */version*, */metadata*, identidade permanente e serviço systemd. **Validado**: executa tarefas locais e mantém os resultados persistidos mesmo sem conectividade.

Fase 3 — Orquestração com RabbitMQ

Pacote de comunicação AMQP: declaração da topologia (exchanges/filas), **publisher confirms**, consumidor de comandos e reconexão automática; o agente passa a consumir comandos e a drenar a outbox. **Validado, incluindo o teste de tolerância a falha**: com o RabbitMQ derrubado, os resultados ficaram retidos na outbox (contagem = 3) e foram drenados assim que o broker retornou (contagem = 0) — nenhum resultado perdido.

Fase 4 — Controlador central

API em FastAPI + PostgreSQL: cadastro de probes, destinos e grupos; recebimento de **planos JSON** com **validador** (probes existentes/ativos, destinos autorizados, período mínimo, duração máxima de iperf3, limite de tarefas) e **expansão de malha** (grupos/topologias em tarefas concretas, com o cálculo de $n(n-1)$); publicação das tarefas no RabbitMQ e auditoria. **Validado**: um plano criado gerou 3 tarefas (ICMP/HTTP/DNS), publicadas e executadas pelo probe.

Fase 5 — Ingestão e visualização

Serviço de ingestão que consome os resultados do RabbitMQ e grava no PostgreSQL/TimescaleDB (**hypertables**: tabela comum de execuções + tabelas tipadas por medição). Grafana provisionado automaticamente com dashboards (RTT médio, perda, TTFB de HTTP, tempo de DNS) e a **matriz Origem × Destino** com células coloridas por RTT/perda. **Validado**: os dados chegam ao banco e os painéis renderizam corretamente.

6. Estado atual vs. resultado mínimo esperado

Item	Situação
Medições ativas (ICMP, DNS, HTTP/HTTPS, traceroute, vazão)	Concluído
Planos declarativos em JSON (validação + expansão de malha)	Concluído
Orquestração assíncrona via RabbitMQ (publisher confirms)	Concluído
Persistência local tolerante a falha (outbox SQLite)	Concluído
Consolidação central em PostgreSQL/TimescaleDB	Concluído
Dashboards temporais + matriz probe × destino (Grafana)	Concluído
Suporte a pares, grupos e malha $n(n-1)$ na expansão	Parcial
Vários probes homogêneos operando em malha	Fase 6

7. Ambiente e reprodutibilidade

O desenvolvimento e os testes foram feitos em **WSL2/Ubuntu**, que replica o ambiente Linux dos Raspberry Pi (o agente usa chamadas de sistema e ferramentas de Linux). A infraestrutura de apoio sobe com um único *docker compose up -d* (RabbitMQ, PostgreSQL/TimescaleDB e Grafana). O agente compila com Go 1.26 e faz *cross-compile* para ARM64; o controlador roda com Python 3.14 (FastAPI/Uvicorn). O repositório é um monorepo no GitHub, com os módulos *contracts*, *prototypes*, *agent*, *controller* e *infra*, além do plano e do diário de bordo.

8. Próximos passos

- **Fase 6:** operar vários probes homogêneos em malha e validar a expansão $n(n-1)$ em escala.
- **Controle de concorrência:** reserva de origem/destino e serialização dos testes de iperf3 (medições intrusivas).
- **Scheduler automático** no controlador (executar os planos por *period_seconds*, sem acionação manual).
- **Agregações contínuas** (5 min, 1 h, 1 dia) no TimescaleDB para consultas históricas mais rápidas.
- **Implantação nos Raspberry Pi reais** (o cross-compile arm64 já está pronto).
- **Início da escrita do TCC**, aproveitando as evidências e decisões já registradas no diário de bordo.